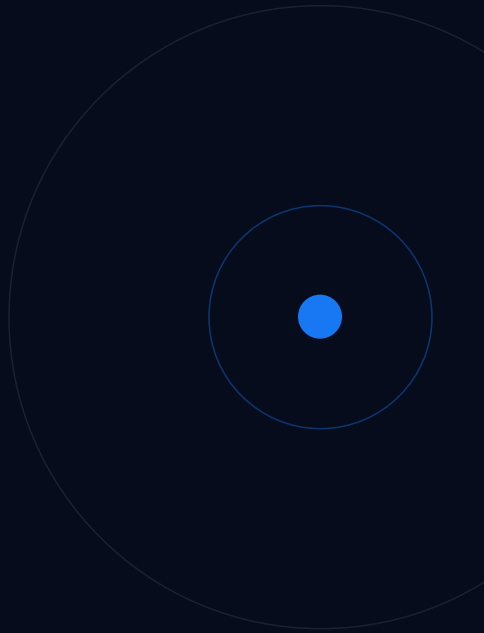
CORE CONCEPTS, PATTERNS, AND TRADEOFFS

# LangGraph & Strands Agents



JOSÉ ORTIZ & ISABEL MORA

PYCON · 2026

# Agenda

01 Introduction

02 The Basics

03 Complex Patterns

04 Strengths & Tradeoffs

05 Takeaways

# 01 Introduction

# Whatsapp group

LangGraph and Strands Agents

WhatsApp group

# About us

**José Ortiz**
ML Engineer @ Loka

**Isabel Mora**
ML Engineer @ Loka

What Is

# LOKA

aws

PARTNER OF THE YEAR

WINNER

2024

# This is About Industry-Level Know-How

What have we implemented using these agent frameworks?

**Strands**

Researchers querying genomic data, biological databases, and research papers from one interface.

➔ Built with Strands so the agent decides which sources to query based on each question.

**LangGraph**

**60.000 emails per month**, around Health Insurance process in the USA.

➔ Now handled with multi-step LangGraph containing business logic

# Why Agent Frameworks?

### Industry shift from chatbots to agents

LLMs can now plan, use tools, and take actions. A framework gives this behavior structure.

### Production agents are complex

Tool routing, memory, error handling, multi-step reasoning. You need more than a prompt.

### The framework is an architectural decision

It shapes how much control you keep, how debuggable your system is, and how fast you can iterate.

LOKA

# Two Different Philosophies

Who controls the workflow?

## Strands Agents

Model-driven

- → You describe the tools, the agent figures out the path
- → Flexible, minimal setup, less predictable.

## LangGraph

Developer-driven

- → You draw the map, the agent follows it
- → Every step is explicit and predictable

# What we will build 💡

**GitHub Repo:** isabelmorar/pycon2026-langgraph-and-strands

# 02 The Basics

# Strands Core Concepts

## Agent = The Orchestrator

➔ Wraps the model, tools, and instructions. The entry point for everything.

## Tools = Capabilities

➔ Functions the model can call, described by their docstring.

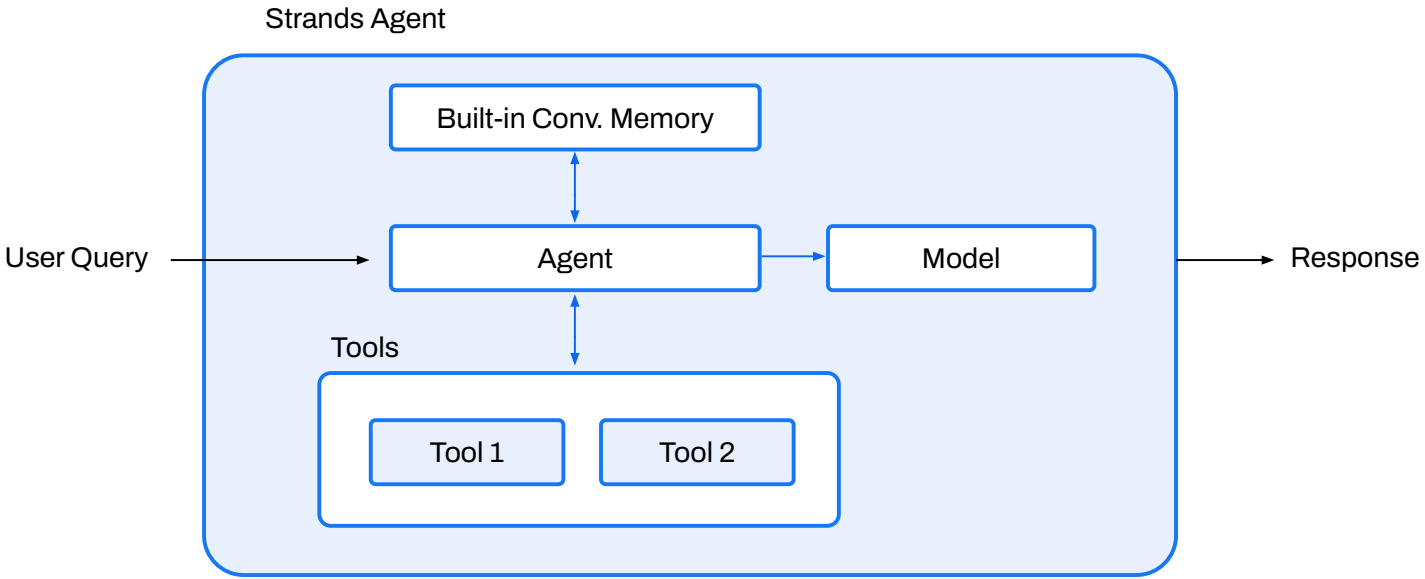
## Instructions = Behavior

➔ Prompt that shapes how the model reasons, responds, and uses its tools.

## Execution = Asking

➔ The agent is invoked with a question. It plans, calls tools, and returns an answer.

# Strands Example



# Hands-On #1.1

**GitHub Repo:** isabelmorar/pycon2026-langgraph-and-strands

# LangGraph Core Concepts

## Nodes = Steps

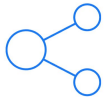
➔ Functions that do the work: call the model, run a tool, transform data, etc.

## Edges = Transitions

➔ Define what runs next. Can be fixed or conditional.

## State = Memory

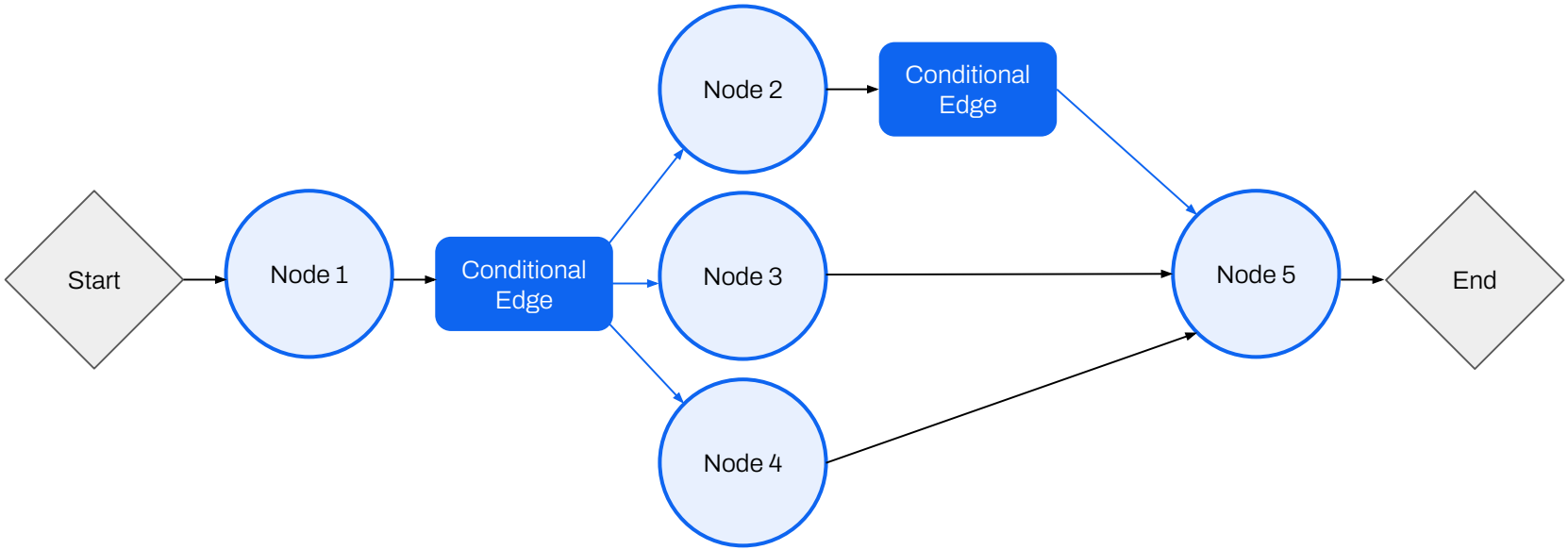
➔ Everything the agent knows, passed from step to step.



## Graph = Blueprint

➔ The full assembly of nodes and edges, compiled into a runnable agent.

# LangGraph Example



# Hands-On #1.2

**GitHub Repo:** isabelmorar/pycon2026-langgraph-and-strands

LOKA

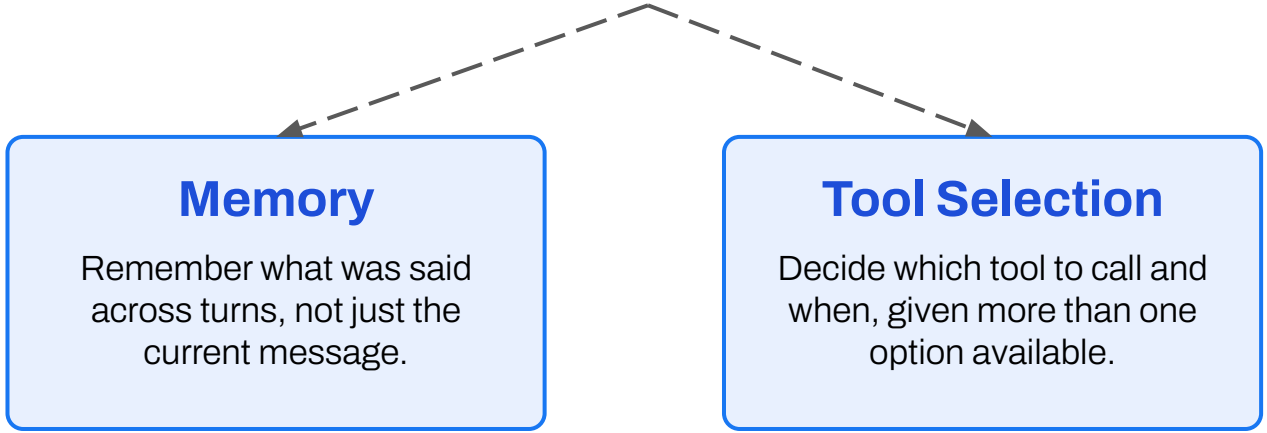
# Initial Comparison

| | Strands Agents | LangGraph |
|---|---|---|
| **Lines of Code** | ~4 | ~16 |
| **Control** | The model decides | You define every step |
| **Visibility** | Reasoning loop is opaque | Every node is inspectable |
| **Boilerplate** | Almost none | Explicit setup required |
| **Testability** | Harder to test in isolation | Each node is a pure function |

# 03 Complex Patterns

# Conceptual Overview

What does a production agent really need?



# Framework Comparison

## Strands

### Memory

The framework manages history for you **automatically.** Conversation managers give additional configuration

### Tool Selection

**Register all tools, model decides**

No routing code. The model reads docstrings and picks at runtime.

## LangGraph

### Memory

`MessagesState + add_messages`

Conversation history lives in state. It is explicit, typed and fully inspectable.

### Tool Selection

**Conditional edge + router function**

You write the routing logic and the model output decides which node runs next.

# Hands-On #2

**GitHub Repo:** isabelmorar/pycon2026-langgraph-and-strands

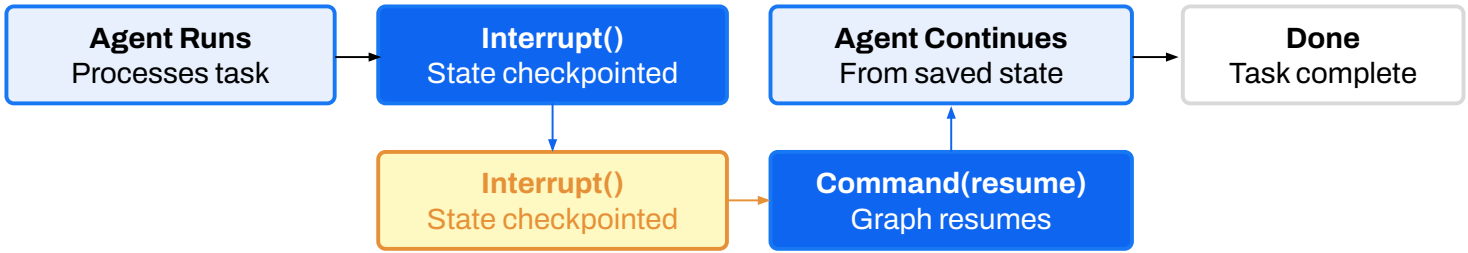
# 04 Strengths and Tradeoffs

# Human-in-the-loop

The agent pauses before taking an action and waits for a human to approve, reject, or redirect.

**Strands**: No native support. Requires exiting the agent, collecting input externally, and re-invoking with the updated context.

**LangGraph:** Built-in support. Execution pauses at a defined point, state is saved automatically, and the agent resumes exactly where it left off once the human responds.

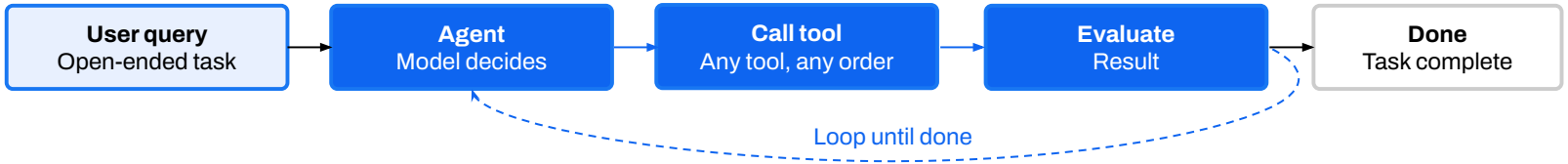


# Open-Ended Reasoning

A pattern where the agent autonomously decides which steps to take, which tools to call, and how many times, without a predefined execution path.

✗ **LangGraph:** Every step must be defined explicitly in advance. Open-ended behavior requires building a meta-agent loop, which adds significant complexity.

✓ **Strands:** Natural fit. The model drives the loop, decides which tools to call and in what order, and iterates until the task is done. No routing code needed.



# Hands-On #3

**GitHub Repo:** isabelmorar/pycon2026-langgraph-and-strands

# 05 Takeaways

# Closing the gaps, where is this going?

## LangChain v1.0 release introduced

`create_agent`

**“We’re leaning hard into three things for LangChain 1.0:**

Our new create_agent abstraction: the fastest way to build an agent with any moadel provider.
Built on the LangGraph runtime, helping to power reliable agents.
Prebuilt and user defined middleware enable step by step control and customization.”

## Strands v1.0 release introduced

**Multi-agent Patterns**

- ➔ **Graph:** A structured, developer-defined flowchart where an agent decides the path
- ➔ **Swarm:** Dynamic team of agents that hand off tasks autonomously
- ➔ **Workflow:** Sequential pipeline with defined steps and handoffs

# Key Points to Remember

**01** LangGraph is better for workflows in which a set of deterministic and repetitive steps is defined. Also, it comes as a more robust, mature and rich framework.

**02** Strands is better for workflows where flexibility per case is required. It has more seamless integrations with AWS resources

# Key Points to Remember

01 LangGraph is better for workflows in which a set of deterministic and repetitive steps is defined. Also, it comes as a more robust, mature and rich framework.

02 Strands is better for workflows where flexibility per case is required. It has more seamless integrations with AWS resources

03 **Loka is a great company with a fun, warm and tech-savvy team!**

# Any Questions?

## Rate this workshop!

## Appy to Loka! 🎯

https://www.loka.com/careers